

Avaliação — Funções e Modularização: O Gestor Acadêmico Modular

1) TAREFA 1 — Investigação Técnica

Nesta etapa, o código foi analisado como um arquiteto de software analisaria: primeiro a lógica foi lida e a saída foi prevista manualmente, sem execução; em seguida, o programa foi executado na IDE para confirmar (ou não) a predição.

1.1 Predição (sem executar o código)

Dados de entrada — Aluno: Maria | Notas: 8.5, 9.0, 7.5, 10.0

Item	Resposta prevista
Média exata	8.75 (35.0 ÷ 4)
Conceito final	B (pois $8.75 \geq 7$ e < 9)
Função coordenadora	relatorio(nome, notas) — é a única função que integra e orquestra todas as demais

1.2 Execução e observação

Ao rodar o programa com os dados fornecidos, o console exibiu:

```
Aluno: Maria
Média: 8.7
Maior nota: 10.0
Menor nota: 7.5
Conceito: B
```

A predição foi confirmada. A média exata calculada internamente é 8.75, mas o console mostra 8.7 porque o comando de formatação `:.1f` arredonda apenas a exibição para uma casa decimal — o valor usado internamente pelas demais funções continua sendo o resultado completo da divisão.

1.3 Questões de pesquisa

Funções com return e destino do valor. Utilizam `return` as funções `media()`, `maior_nota()`, `menor_nota()` e `conceito()`. O valor devolvido por cada uma é capturado em variáveis locais dentro de `relatorio()` (`m`, `maior`, `menor` e `c`), que depois são usadas nos prints. Sem o `return`, o resultado do cálculo ficaria restrito ao escopo interno da função e se perderia assim que ela terminasse.

Por que `relatorio()` não possui `return`. Porque a responsabilidade dessa função é exibir informações na tela (efeito colateral de entrada/saída), e não produzir um valor para ser reaproveitado por outra função. Caso se tente imprimir o resultado dela — `print(relatorio(...))` —, o Python exibirá `None`, que é o valor padrão devolvido implicitamente por qualquer função sem um `return` explícito.

Vantagem de usar media(notas) em vez do cálculo direto. Isolar o cálculo em uma função própria traz reutilização (outras partes do sistema podem chamar media() sem repetir a fórmula), manutenção centralizada (uma eventual mudança na regra de cálculo é feita em um único lugar), legibilidade (o nome media(notas) já é autoexplicativo dentro de relatorio()) e responsabilidade única — media() cuida exclusivamente do cálculo, enquanto relatorio() cuida exclusivamente da exibição.

Em resumo, o código já nasce bem modularizado: cada função tem uma responsabilidade coesa e baixo acoplamento entre si, e relatorio() funciona como a coordenadora que organiza o trabalho das funções especialistas (media, maior_nota, menor_nota e conceito).

2) TAREFA 2 — Expansão e Criação de Funcionalidade

Para atender aos novos requisitos da coordenação pedagógica, o código original foi expandido mantendo o mesmo princípio de responsabilidade única de cada função. As alterações foram organizadas em blocos comentados no arquivo sistema_academico.py, cujas capturas de tela seguem em anexo, na ordem em que aparecem no código.

2.1 Novas funções de apoio

situacao(media_aluno). Recebe a média já calculada e devolve "Aprovado" quando ela é maior ou igual a 6.0, e "Reprovado" caso contrário — seguindo a mesma lógica de conceito(), com um único ponto de decisão e retorno direto.

quantidade_acima_media(notas, media_aluno). Percorre a lista de notas com um laço for e usa um contador que é incrementado toda vez que uma nota individual é maior que a média do aluno; ao final, o contador é devolvido com return. Essa função reaproveita a média já calculada por media(), em vez de recalculá-la, evitando processamento redundante.

2.2 Atualização da função coordenadora

A função relatorio() foi atualizada para chamar situacao(m) e quantidade_acima_media(notas, m) logo após calcular a média, o conceito, a maior e a menor nota, e passou a exibir também a situação (Aprovado/Reprovado) e a quantidade de notas acima da média do próprio aluno. Ela continua sem return, pois sua responsabilidade permanece sendo apenas a exibição formatada do relatório individual.

2.3 Relatório da turma (programa principal)

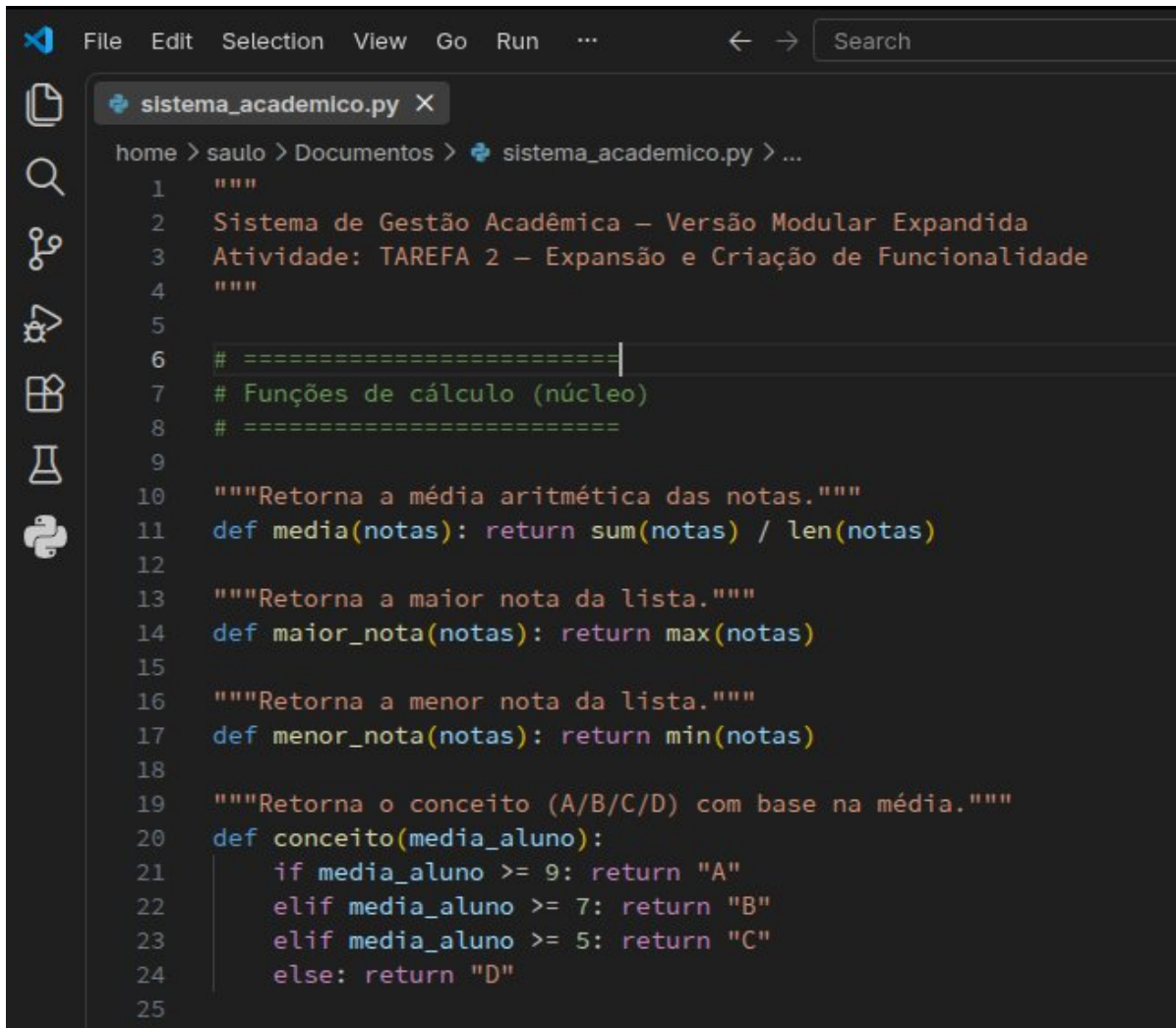
O programa principal foi reescrito dentro de uma função main(), protegida pelo bloco if __name__ == "__main__", para permitir processar vários alunos em um laço while True. A cada iteração é solicitado o nome do aluno, até que o usuário digite "FIM" (verificado com nome.upper() == "FIM", tornando a checagem independente de maiúsculas/minúsculas). Para cada aluno, há também um laço interno de validação da entrada das notas, com try/except ValueError, garantindo que apenas números válidos e uma lista não vazia sejam aceitos antes de gerar o relatório individual.

Durante o laço, o nome e a média de cada aluno são armazenados nas listas nomes_turma e medias_turma. Ao final (quando o usuário digita "FIM"), essas listas são usadas para calcular as estatísticas da turma: o total de alunos (len(nomes_turma)), a média geral da turma (média das médias individuais) e as médias global máxima e mínima, obtidas com max() e min() sobre medias_turma. Para identificar os nomes correspondentes a essas médias — inclusive em caso de empate entre dois ou mais alunos — foi usada uma list comprehension combinada com zip(nomes_turma, medias_turma), que percorre nome e média em pares e seleciona todos os nomes cuja média é igual à maior (ou à menor) média global.

Por fim, um bloco `if nomes_turma: else:` trata o caso em que nenhum aluno chega a ser cadastrado (usuário digita "FIM" imediatamente), evitando um erro de divisão por zero ou de `max()/min()` em lista vazia, e exibindo a mensagem "Nenhum aluno foi registrado." nesse cenário.

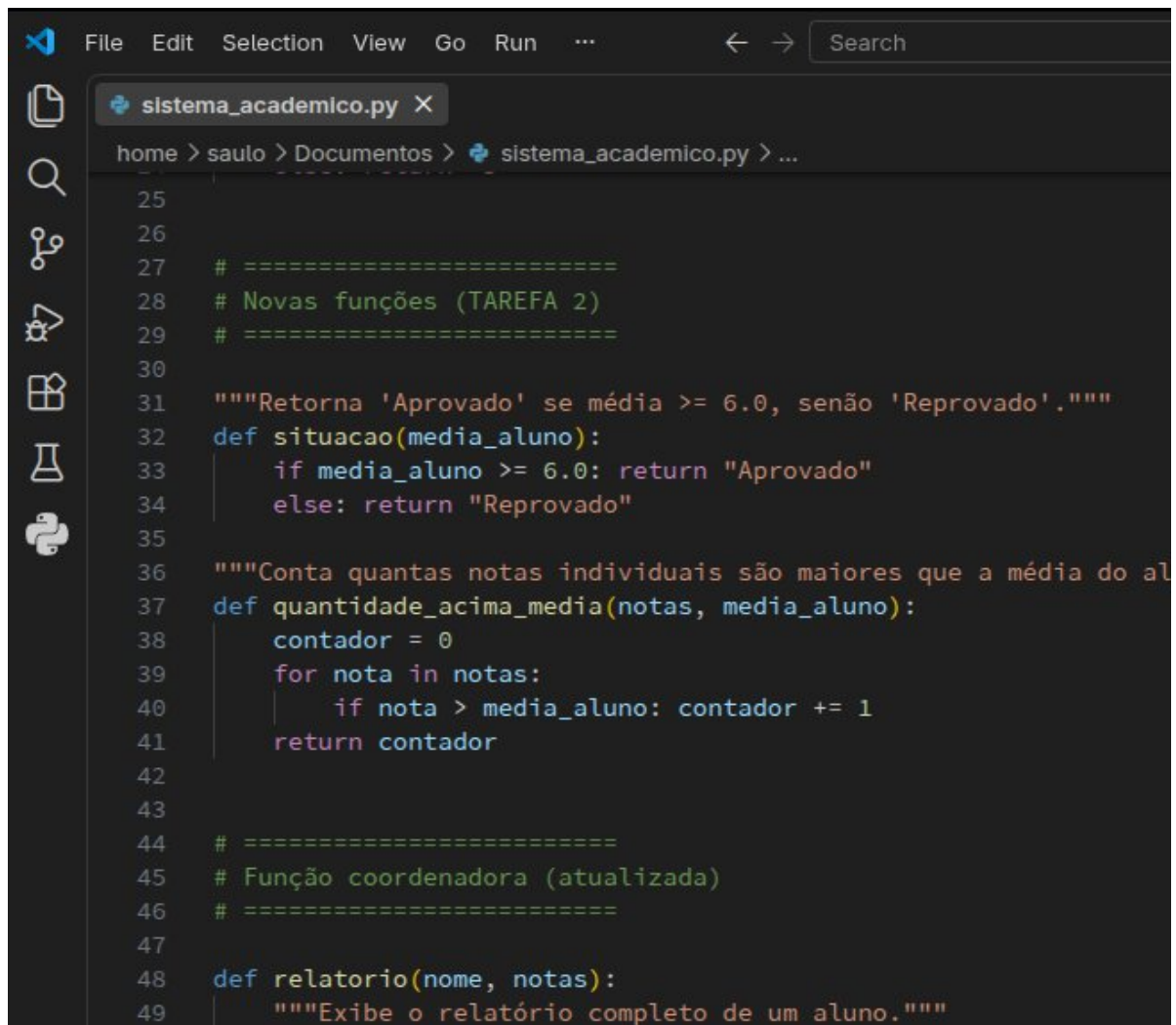
Anexo — Código-fonte (sistema_academico.py)

As capturas de tela a seguir mostram o arquivo completo, na ordem das linhas de código, do início ao fim.



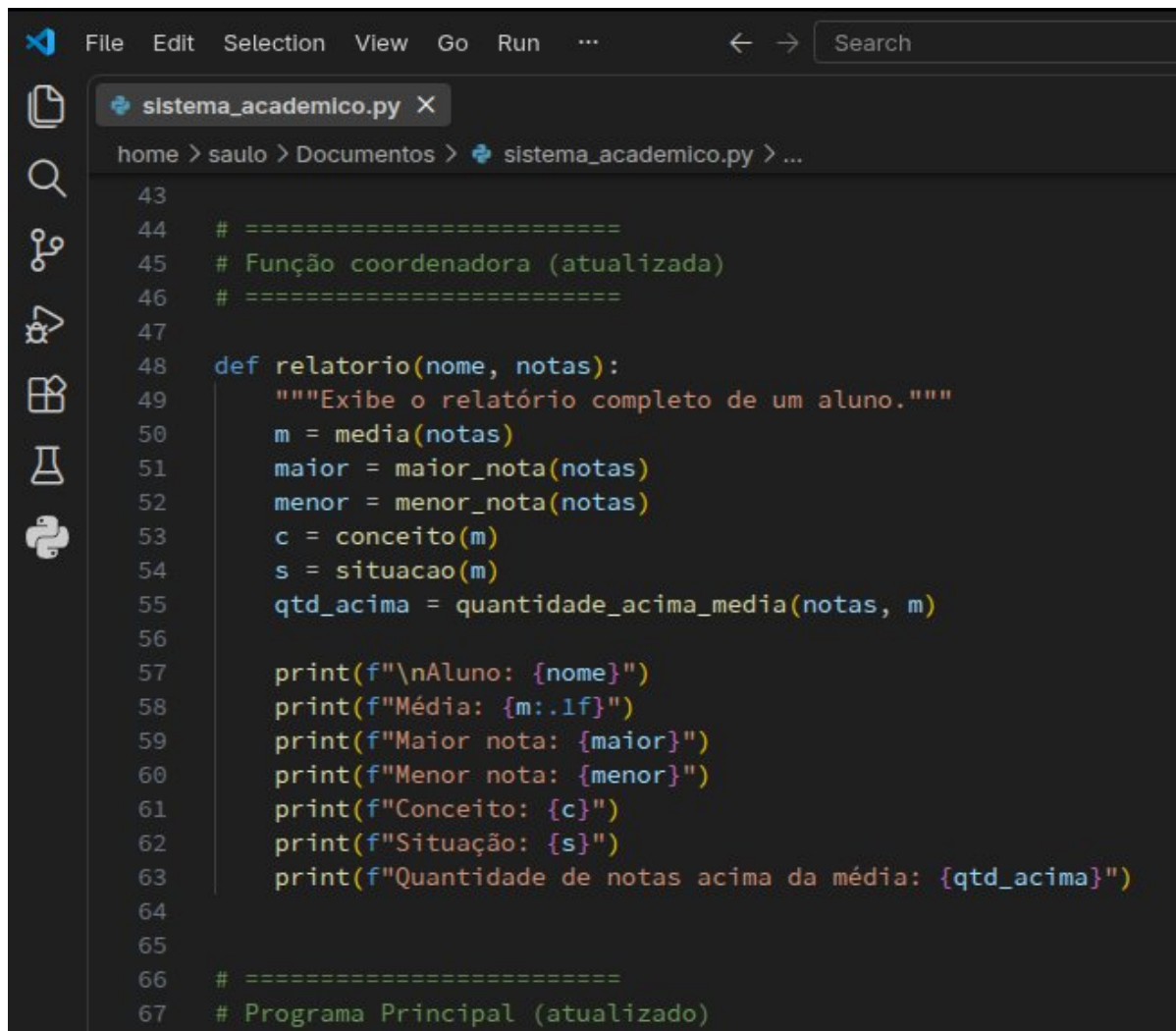
```
1  """
2  Sistema de Gestão Acadêmica – Versão Modular Expandida
3  Atividade: TAREFA 2 – Expansão e Criação de Funcionalidade
4  """
5
6  # =====
7  # Funções de cálculo (núcleo)
8  # =====
9
10 """Retorna a média aritmética das notas."""
11 def media(notas): return sum(notas) / len(notas)
12
13 """Retorna a maior nota da lista."""
14 def maior_nota(notas): return max(notas)
15
16 """Retorna a menor nota da lista."""
17 def menor_nota(notas): return min(notas)
18
19 """Retorna o conceito (A/B/C/D) com base na média."""
20 def conceito(media_aluno):
21     if media_aluno >= 9: return "A"
22     elif media_aluno >= 7: return "B"
23     elif media_aluno >= 5: return "C"
24     else: return "D"
25
```

Núcleo de funções de cálculo (linhas 1–25): `media`, `maior_nota`, `menor_nota` e `conceito`.



```
25
26
27 # =====
28 # Novas funções (TAREFA 2)
29 # =====
30
31 """Retorna 'Aprovado' se média >= 6.0, senão 'Reprovado'."""
32 def situacao(media_aluno):
33     if media_aluno >= 6.0: return "Aprovado"
34     else: return "Reprovado"
35
36 """Conta quantas notas individuais são maiores que a média do al
37 def quantidade_acima_media(notas, media_aluno):
38     contador = 0
39     for nota in notas:
40         if nota > media_aluno: contador += 1
41     return contador
42
43
44 # =====
45 # Função coordenadora (atualizada)
46 # =====
47
48 def relatorio(nome, notas):
49     """Exibe o relatório completo de um aluno."""
```

Novas funções da Tarefa 2 (linhas 26-43): `situacao()` e `quantidade_acima_media()`.



```
43
44 # =====
45 # Função coordenadora (atualizada)
46 # =====
47
48 def relatorio(nome, notas):
49     """Exibe o relatório completo de um aluno."""
50     m = media(notas)
51     maior = maior_nota(notas)
52     menor = menor_nota(notas)
53     c = conceito(m)
54     s = situacao(m)
55     qtd_acima = quantidade_acima_media(notas, m)
56
57     print(f"\nAluno: {nome}")
58     print(f"Média: {m:.1f}")
59     print(f"Maior nota: {maior}")
60     print(f"Menor nota: {menor}")
61     print(f"Conceito: {c}")
62     print(f"Situação: {s}")
63     print(f"Quantidade de notas acima da média: {qtd_acima}")
64
65
66 # =====
67 # Programa Principal (atualizado)
```

Função coordenadora relatorio() atualizada (linhas 44–68), exibindo conceito, situação e notas acima da média.


```
File Edit Selection View Go Run ... Search
sistema_academico.py X
home > saulo > Documentos > sistema_academico.py > ...
65
66 # =====
67 # Programa Principal (atualizado)
68 # =====
69
70 def main():
71     # Listas para armazenar os dados da turma para estatísticas finais
72     nomes_turma = []
73     medias_turma = []
74
75     print("=" * 50)
76     print(" SISTEMA DE GESTÃO ACADÊMICA ")
77     print(" Digite 'FIM' no nome para encerrar ")
78     print("=" * 50)
79
80     while True:
81         nome = input("\nDigite o nome do aluno (ou 'FIM' para encerrar): ").strip().capitalize()
82
83         if nome.upper() == "FIM": break
84
85         # Validação simples: garanta que haja ao menos 1 nota
86         while True:
87             entrada = input("Digite as notas separadas por espaço: ").strip()
88             try:
89                 notas = list(map(float, entrada.split()))
90                 if len(notas) == 0:
91                     raise ValueError
92                 break
93             except ValueError:
94                 print("⚠ Entrada inválida. Digite ao menos uma nota numérica.")
95
```

Início da função main() (linhas 65–95): laço de coleta de alunos e validação das notas.

```
File Edit Selection View Go Run ... Search
sistema_academico.py X
home > saulo > Documentos > sistema_academico.py > ...
70 def main():
96     # Gera o relatório do aluno
97     relatorio(nome, notas)
98
99     # Armazena para as estatísticas finais
100     nomes_turma.append(nome)
101     medias_turma.append(media(notas))
102
103     # =====
104     # Estatísticas finais da turma
105     # =====
106     if nomes_turma: # só calcula se houver ao menos 1 aluno
107         media_geral = sum(medias_turma) / len(medias_turma)
108         maior_media_global = max(medias_turma)
109         menor_media_global = min(medias_turma)
110
111         # Descobre todos os alunos que empataram na maior e na menor média
112         nomes_maior = [n for n, m in zip(nomes_turma, medias_turma) if m == maior_media_global]
113         nomes_menor = [n for n, m in zip(nomes_turma, medias_turma) if m == menor_media_global]
114
115         print("\n" + "=" * 50)
116         print(" 📊 ESTATÍSTICAS DA TURMA ")
117         print("=" * 50)
118         print(f"Total de alunos: {len(nomes_turma)}")
119         print(f"Média geral da turma: {media_geral:.2f}")
120         print(f"Maior média global: {maior_media_global:.1f} → Aluno(s): {'', '.join(nomes_maior)}")
121         print(f"Menor média global: {menor_media_global:.1f} → Aluno(s): {'', '.join(nomes_menor)}")
122     else:
123         print("\nNenhum aluno foi registrado.")
124
125     print("\nEncerrando o sistema. Até logo! 📢")
```

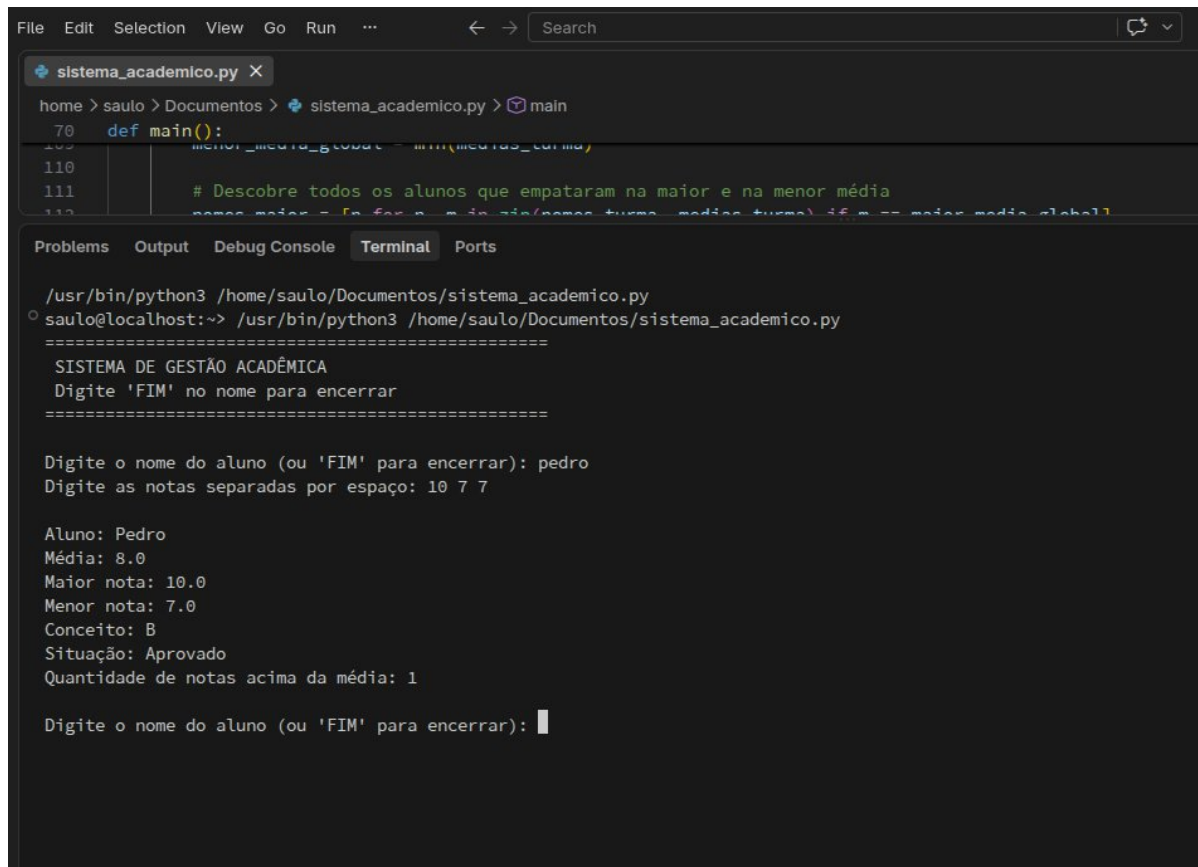
Continuação de main() (linhas 96–125): estatísticas finais da turma.

```
File Edit Selection View Go Run ... Search
sistema_academico.py X
home > saulo > Documentos > sistema_academico.py > main
109 def main():
110     menor_media_global = min(medias_turma)
111     # Descobre todos os alunos que empataram na maior e na menor média
112     nomes_maior = [n for n, m in zip(nomes_turma, medias_turma) if m == maior_media_global]
113     nomes_menor = [n for n, m in zip(nomes_turma, medias_turma) if m == menor_media_global]
114
115     print("\n" + "-" * 50)
116     print(" ▯ ESTATÍSTICAS DA TURMA ")
117     print("-" * 50)
118     print(f"Total de alunos: {len(nomes_turma)}")
119     print(f"Média geral da turma: {media_geral:.2f}")
120     print(f"Maior média global: {maior_media_global:.1f} → Aluno(s): {' '.join(nomes_maior)}")
121     print(f"Menor média global: {menor_media_global:.1f} → Aluno(s): {' '.join(nomes_menor)}")
122 else:
123     print("\nNenhum aluno foi registrado.")
124
125 print("\nEncerrando o sistema. Até logo!")
126
127
128 # Ponto de entrada do programa
129 if __name__ == "__main__":
130     main()
131
```

Encerramento de main() e ponto de entrada do programa (linhas 109–131).

Anexo B — Execução do Programa

As capturas abaixo mostram o terminal com o programa em execução, evidenciando o funcionamento das novas funcionalidades da Tarefa 2: relatório individual expandido (situação e quantidade de notas acima da média) e o relatório final da turma.



The screenshot shows a code editor with a file named `sistema_academico.py` and a terminal window below it. The terminal output shows the program's execution for a student named Pedro with grades 10, 7, and 7.

```
File Edit Selection View Go Run ... Search
sistema_academico.py X
home > saulo > Documentos > sistema_academico.py > main
79 def main():
109     menor_media_global = min(media_turma)
110
111     # Descobre todos os alunos que empataram na maior e na menor média
112     nomes_maiores = [n for n, m in zip(nomes_turma, medias_turma) if m == maior_media_global]

Problems Output Debug Console Terminal Ports

/usr/bin/python3 /home/saulo/Documentos/sistema_academico.py
saulo@localhost:~> /usr/bin/python3 /home/saulo/Documentos/sistema_academico.py
=====
SISTEMA DE GESTÃO ACADÊMICA
Digite 'FIM' no nome para encerrar
=====

Digite o nome do aluno (ou 'FIM' para encerrar): pedro
Digite as notas separadas por espaço: 10 7 7

Aluno: Pedro
Média: 8.0
Maior nota: 10.0
Menor nota: 7.0
Conceito: B
Situação: Aprovado
Quantidade de notas acima da média: 1

Digite o nome do aluno (ou 'FIM' para encerrar):
```

Execução para o aluno Pedro (notas 10, 7, 7): relatório individual exibindo Conceito, Situação e Quantidade de notas acima da média.


```
File Edit Selection View Go Run ... Search
sistema_academico.py X
home > saulo > Documentos > sistema_academico.py > main
70 def main():
109     menor_media_global = min(media_turma)
110
111     # Descubra todos os alunos que empataram na maior e na menor média
112     nomes_maior = [n for n, m in zip(nomes_turma, medias_turma) if m == maior_media_global]

Problems Output Debug Console Terminal Ports
saulo@localhost:~> /usr/bin/python3 /home/saulo/Documentos/sistema_academico.py

Digite o nome do aluno (ou 'FIM' para encerrar): Angelica
Digite as notas separadas por espaço: 8 9 7

Aluno: Angelica
Média: 8.0
Maior nota: 9.0
Menor nota: 7.0
Conceito: B
Situação: Aprovado
Quantidade de notas acima da média: 1

Digite o nome do aluno (ou 'FIM' para encerrar): fim

=====
>>> ESTATÍSTICAS DA TURMA <<<
=====
Total de alunos: 2
Média geral da turma: 8.00
Maior média global: 8.0 → Aluno(s): Pedro, Angelica
Menor média global: 8.0 → Aluno(s): Pedro, Angelica

Encerrando o sistema. Até logo!
saulo@localhost:~>
```

Execução com dois alunos (Pedro e Angelica), encerrada com "FIM": relatório da turma com total de alunos, média geral e alunos com maior/menor média global.